pyLGC — Python Interface for LGC2

Least-Squares Geodetic Computation

Jürgen Gutekunst

April 2, 2026

Contents

1 Introduction

`pyLGC` is a Python module that provides access to the LGC2 least-squares adjustment engine. It loads a native shared library (`pyLGC_C.dll` on Windows, `libpyLGC_C.so` on Linux) via `ctypes`.

The module exposes the evaluator, which encapsulates the linearised least-squares problem built from an LGC input file, and allows the user to inspect, perturb, evaluate, and solve the underlying system. For a detailed description of the underlying concepts and methods, see [1].

This document is structured as follows: Section 2 describes the underlying least-squares problem, Section 3 documents the Python API, and Section 4 provides examples of common workflows.

2 The Least-Squares Problem

This section summarises the nonlinear estimation problem solved by LGC2. The formulation follows [2]; see also [1] for a broader overview and [3] for the general theory of least squares.

In the general theory of geodetic adjustment, the observation model can take the implicit (combined) form $F(x, L+v) = 0$, where both parameters and observations appear inside F. In LGC2, however, all observation models are of *parametric* form: $f(x) - (L+v) = 0$, i.e. the model predicts the observations as a function of the parameters alone. As a consequence, the matrix $B = \partial F/\partial L$ is always $-I$ (the negative identity), which simplifies the solution considerably.

2.1 The Nonlinear Estimation Problem

Given a parametric observation model $f\colon \mathbb{R}^{n_x} \to \mathbb{R}^{n_L}$ that predicts measurement values from unknown parameters, LGC2 solves the constrained nonlinear least-squares problem

$$\begin{aligned}
&\min\ \tfrac{1}{2}\, v^T P\, v \\
&\text{over } (x, v) \in \mathbb{R}^{n_x} \times \mathbb{R}^{n_L}, \\
&\text{subject to} \\
&\quad f(x) + B\, v - L = 0, \\
&\quad C(x) = 0,
\end{aligned} \tag{1}$$

where $x \in \mathbb{R}^{n_x}$ is the parameter vector, $L \in \mathbb{R}^{n_L}$ the measurement vector, $v \in \mathbb{R}^{n_L}$ the residuals, $B \in \mathbb{R}^{n_L \times n_L}$ the observation coefficient matrix, $C\colon \mathbb{R}^{n_x} \to \mathbb{R}^{n_C}$ the constraint function, and $P \in \mathbb{R}^{n_L \times n_L}$ the weight matrix. Since all observation models in LGC2 are of parametric form $f(x) - (L+v) = 0$, the matrix B is always $-I$ (negative identity). The API nevertheless exposes B and B^{-1} for completeness.

2.2 Gauss–Newton Linearisation

Problem (1) is solved by the Gauss–Newton algorithm. At iterate x_k, we define the misclosure vector as

$$W_k = f(x_k) - L \in \mathbb{R}^{n_L}, \tag{2}$$

which measures how well the current parameters explain the measurements. Linearising f around x_k gives

$$f(x_k + \Delta x) \approx f(x_k) + A_k\, \Delta x, \qquad A_k = \frac{\partial f}{\partial x}(x_k) \in \mathbb{R}^{n_L \times n_x}. \tag{3}$$

Similarly, the constraint is linearised as

$$C(x_k + \Delta x) \approx C(x_k) + A_{2,k}\, \Delta x, \qquad A_{2,k} = \frac{\partial C}{\partial x}(x_k) \in \mathbb{R}^{n_C \times n_x}. \tag{4}$$

Substituting the linearised model into the observation equation $f(x) + B\,v - L = 0$ gives

$$f(x_k)+A_k\,\Delta x+B\,v-L \;\approx\; 0 \qquad\Longrightarrow\qquad v \;\approx\; -B^{-1}\big(f(x_k)-L+A_k\,\Delta x\big) \;=\; -B^{-1}(W_k+A_k\,\Delta x). \tag{5}$$

With $B = -I$ this simplifies to $v \approx W_k + A_k\,\Delta x$. Inserting into the objective $\min \frac{1}{2}\,v^T P\,v$ yields the linearised subproblem

$$\begin{aligned} &\min \tfrac{1}{2}\,(W_k + A_k\,\Delta x)^T P\,(W_k + A_k\,\Delta x) \quad \text{over } \Delta x,\\ &\text{s.t.} \quad A_{2,k}\,\Delta x + C(x_k) = 0. \end{aligned} \tag{6}$$

This defines the matrices and vectors accessible through the `pyLGC` API:

$$A = \frac{\partial f}{\partial x}(x_k) \in \mathbb{R}^{n_L \times n_x}, \qquad \text{(design / Jacobian matrix)} \tag{7}$$

$$B = -I \in \mathbb{R}^{n_L \times n_L}, \qquad \text{(observation coefficient matrix)} \tag{8}$$

$$W = f(x_k) - L \in \mathbb{R}^{n_L}, \qquad \text{(misclosure vector)} \tag{9}$$

$$A_2 = \frac{\partial C}{\partial x}(x_k) \in \mathbb{R}^{n_C \times n_x}, \qquad \text{(constraint Jacobian)} \tag{10}$$

$$W_2 = C(x_k) \in \mathbb{R}^{n_C}, \qquad \text{(constraint misclosure)} \tag{11}$$

$$P \in \mathbb{R}^{n_L \times n_L}. \qquad \text{(weight matrix)} \tag{12}$$

2.3 Solution of the Linearised Problem

Setting the gradient of the Lagrangian of (6) to zero yields the KKT system

$$\begin{pmatrix} A^T P\,A & A_2^T \\ A_2 & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \lambda \end{pmatrix} = -\begin{pmatrix} A^T P\,W \\ W_2 \end{pmatrix}, \tag{13}$$

where λ is the vector of Lagrange multipliers for the constraints. In the unconstrained case ($n_C = 0$), this simplifies to the normal equations

$$A^T P\,A\,\Delta x = -A^T P\,W. \tag{14}$$

The parameters are then updated as $x_{k+1} = x_k + \Delta x$ and the process repeats until convergence. The residuals at convergence are $v = f(x^*) - L = W^*$.

3 API Reference

3.1 Architecture

`pyLGC` is structured in two layers:

1. `pyLGC_C.dll / libpyLGC_C.so` — A shared library with a plain C API (`extern "C"`) wrapping the C++ LGC2 engine. It has no dependency on Python and can be called from any language with a C foreign-function interface.

2. `pyLGC.py` — A pure-Python module that loads the shared library via `ctypes` and exposes the `Evaluator`, `AdjustablePoint`, and `AdjustableFrame` classes. Arrays are returned as `numpy` arrays for efficient data transfer. Requires `numpy`; `scipy` is optional (for sparse matrix construction).

Because the shared library has no compile-time dependency on Python headers, it can be used from any language with a C foreign-function interface.

3.2 Module-Level Types

`UEOIndices` Named tuple holding the four key dimensions of the problem:

Symbol	Field	Meaning
n_x	`UIndex`	Number of unknowns (parameters)
n_L	`EIndex`	Number of equations (= number of observations)
n_L	`OIndex`	Number of observations
n_C	`CIndex`	Number of constraints

3.3 Class `Evaluator`

The central class. Each instance encapsulates one least-squares problem loaded from an LGC input file.

Constructor

`Evaluator(filePath: str)`
Load and parse the given LGC input file. Raises `RuntimeError` if the file cannot be opened or contains errors.

Problem Dimensions

`getIndices() -> UEOIndices`
Return the dimensions (n_x, n_L, n_L, n_C) of the problem. Available immediately after construction.

Parameter Management

`getEstParams() -> numpy.ndarray`
Return the current parameter vector $x \in \mathbb{R}^{n_x}$ as a one-dimensional numpy array.

`setParameters(para)`
Set the parameter vector. Accepts any sequence of length n_x. Raises `RuntimeError` if the dimension is wrong. *Note:* after calling `setParameters`, a new `evaluate()` is required before the getters return valid results.

Evaluation

`evaluate() -> bool`
Linearise the functional model at the current parameter values. Computes all matrices and vectors. Returns `True` on success. Must be called before any matrix or vector getter.

Vectors

`getMisclosure() -> numpy.ndarray`
Return the misclosure vector $W = f(x_k) - L \in \mathbb{R}^{n_L}$. Requires a preceding `evaluate()`.

`getConstraintMisclosure() -> numpy.ndarray`
Return the constraint misclosure $W_2 = C(x_k) \in \mathbb{R}^{n_C}$. Requires a preceding `evaluate()`.

Sparse Matrices

All sparse matrix getters return a tuple of five elements:

```
(rows, cols, vals, nrows, ncols)
```

where `rows`, `cols` are integer arrays, `vals` is a float array (COO format), and `nrows`, `ncols` are the matrix dimensions. See Section 4.1 for how to construct `scipy.sparse` matrices from these tuples.

`getAMatrix()`
Design matrix (Jacobian) $A = \partial f/\partial x \in \mathbb{R}^{n_L \times n_x}$.

`getBMatrix()`
Observation coefficient matrix $B \in \mathbb{R}^{n_L \times n_L}$. Since LGC2 uses parametric models, $B = -I$ always (see Section 2).

`getInvBMatrix()`
Inverse $B^{-1} \in \mathbb{R}^{n_L \times n_L}$.

`getA2Matrix()`
Constraint Jacobian $A_2 = \partial C/\partial x \in \mathbb{R}^{n_C \times n_x}$.

`getPMatrix()`
Weight matrix $P \in \mathbb{R}^{n_L \times n_L}$.

All require a preceding `evaluate()`.

Finite Differences

`getFiniteDifferenceA(finiteDiffEpsilon=1e-6) -> numpy.ndarray`
Compute a numerical approximation of the Jacobian A using forward finite differences with step size `finiteDiffEpsilon`. Returns a dense $n_L \times n_x$ numpy array. Useful for verifying analytic derivatives (see Section 4.4). Requires a preceding `evaluate()`.

Solving

`tryLGCSolve() -> (bool, numpy.ndarray)`
Run the full iterative least-squares solution. Returns a tuple `(ok, solution)` where `ok` indicates convergence and `solution` $\in \mathbb{R}^{n_x}$ is the estimated parameter vector. Can be called without a prior `evaluate()`. *Note:* after solving, `evaluate()` must be called again before the matrix/vector getters are valid.

Observation Mapping

`getObsIndexToLineNumber() -> dict`
Return a dictionary mapping internal observation indices to line numbers in the input file. Useful for tracing residuals back to specific observations.

Point and Frame Access

`getPoint(name: str) -> AdjustablePoint`
Look up an adjustable point by name. Raises `RuntimeError` if not found.

`getFrame(name: str) -> AdjustableFrame`
Look up an adjustable reference frame by name. Raises `RuntimeError` if not found.

3.4 Class `AdjustablePoint`

Represents an adjustable point in the network. Instances are obtained via `Evaluator.getPoint()` and are valid as long as the parent `Evaluator` exists.

`getName() -> str`
Return the point name.

`getFirstUidx() -> int`
Return the index of this point's first unknown in the global parameter vector. Together with `getRelativeUnknIndices()`, this identifies which elements of x belong to this point.

`getRelativeUnknIndices() -> list[int]`
Return the list of relative unknown indices (offsets from `getFirstUidx()`). For a fully free 3D point this is `[0, 1, 2]`; for a partially fixed point some components are absent; for a fully fixed point this is an empty list.

`getEstVector() -> list[float]`
Return the current estimated coordinate vector (always 3 elements for a 3D point, regardless of which components are fixed).

3.5 Class `AdjustableFrame`

Represents an adjustable Helmert transformation frame. The interface is identical to `AdjustablePoint`:

`getName() -> str`
Frame name.

`getFirstUidx() -> int`
First unknown index in the global parameter vector.

`getRelativeUnknIndices() -> list[int]`
Relative unknown indices. Up to 7 parameters (3 translations, 3 rotations, 1 scale); fixed components are absent.

`getEstVector() -> list[float]`
Current estimated transformation parameter vector.

3.6 Error Handling

All errors are reported as Python `RuntimeError` exceptions with descriptive messages. Common error conditions:

Situation	Example message
File not found	`Cannot open file: network.lgc2`
Malformed input file	`Errors found in the input file.`
Wrong parameter dimension	`variable has wrong dimension, expected: 13, actual: 3`
Getter before `evaluate()`	`Must call evaluate() before using getters`
Unknown point name	`Point NOPE not found`
Unknown frame name	`Frame NOPE not found`

4 Typical Workflow and Examples

All code listings in this section are available as standalone Python scripts in the `examples/` directory alongside this document.

The standard usage pattern follows a *load* – *set* – *evaluate* – *get* cycle:

1. **Load.** Create an `Evaluator` from an LGC input file. This parses the file, builds the internal data structures, and sets the provisional parameter values.
2. **Set parameters** (optional). Replace the current parameter vector with custom values using `setParameters()`.
3. **Evaluate.** Call `evaluate()` to linearise the functional model at the current parameters. This computes the matrices A, P, A_2 and the misclosure vectors W, W_2.
4. **Get results.** Retrieve any combination of matrices and vectors via the getter methods.
5. **Repeat** steps 2–4 as needed (e.g. for iterative solving or sensitivity analysis).

Alternatively, `tryLGCSolve()` performs the full iterative solution internally, returning the converged parameter vector directly.

Listing 1: Basic workflow example (`examples/01_basic_workflow.py`).

```
import pyLGC

# 1. Load
ev = pyLGC.Evaluator("Title-Example.lgc2")
idx = ev.getIndices()
print(f"Unknowns: {idx.UIndex}, Equations: {idx.EIndex}")

# 2. Evaluate at provisional parameters
ev.evaluate()

# 3. Inspect the linearised system
A = ev.getAMatrix()        # (rows, cols, vals, nrows, ncols)
w = ev.getMisclosure()     # numpy array of length EIndex

# 4. Solve
ok, solution = ev.tryLGCSolve()
if ok:
    # 5. Evaluate at the solution
    ev.setParameters(solution)
    ev.evaluate()
    w_at_solution = ev.getMisclosure()
```

4.1 Sparse Matrix Usage with SciPy

All sparse matrices are returned in COO (coordinate) format as plain arrays, compatible with `scipy.sparse`:

Listing 2: Converting COO tuples to SciPy sparse matrices (`examples/02_sparse_matrices.py`).

```
from scipy.sparse import coo_matrix
import pyLGC

```

```python
ev = pyLGC.Evaluator("Title-Example.lgc2")
ev.evaluate()

# Convert COO tuple to scipy sparse matrix
rows, cols, vals, nr, nc = ev.getAMatrix()
A = coo_matrix((vals, (rows, cols)), shape=(nr, nc)).tocsr()

rows, cols, vals, nr, nc = ev.getPMatrix()
P = coo_matrix((vals, (rows, cols)), shape=(nr, nc)).tocsr()

w = ev.getMisclosure()
```

4.2 Building the Normal Equations in SciPy

Given the matrices from the evaluator, the normal equation (14) can be assembled and solved in Python:

Listing 3: Forming and solving the normal equations (`examples/03_normal_equations.py`).

```python
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import spsolve
import pyLGC

ev = pyLGC.Evaluator("Title-Example.lgc2")
ev.evaluate()

# Build sparse matrices
def to_csr(tup):
    rows, cols, vals, nr, nc = tup
    return coo_matrix((vals, (rows, cols)), shape=(nr, nc)).tocsr()

A = to_csr(ev.getAMatrix())
P = to_csr(ev.getPMatrix())
w = ev.getMisclosure()

# Normal equations: (A^T P A) dx = -A^T P W
N = A.T @ P @ A
n = A.T @ P @ w
dx = spsolve(N, -n)
```

4.3 Gauss–Newton Iteration

The following example implements a simple Gauss–Newton solver using the matrices provided by the evaluator. At each iteration the normal equations (14) are formed and solved, and the parameter vector is updated until the step size falls below a tolerance.

Listing 4: Simple Gauss–Newton solver, validated against the built-in solver (`examples/04_gauss_newton.py`).

```python
import numpy as np
from numpy.linalg import norm
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import spsolve
import pyLGC

def to_csr(tup):
```

```python
    rows, cols, vals, nr, nc = tup
    return coo_matrix((vals, (rows, cols)), shape=(nr, nc)).tocsr()

ev = pyLGC.Evaluator("Title-Example.lgc2")

# 1. Reference solution from the built-in solver
x0 = ev.getEstParams().copy()
ok, x_ref = ev.tryLGCSolve()

# 2. Reset to provisional values and run own Gauss-Newton
x = x0.copy()
for k in range(50):
    ev.setParameters(x)
    ev.evaluate()

    A = to_csr(ev.getAMatrix())
    P = to_csr(ev.getPMatrix())
    w = ev.getMisclosure()

    # Solve normal equations: (A^T P A) dx = -A^T P w
    N = A.T @ P @ A
    dx = spsolve(N, -A.T @ P @ w)
    x = x + dx

    print(f"iter {k:2d}  |dx| = {norm(dx):.2e}  |w| = {norm(w):.2e}")
    if norm(dx, np.inf) < 1e-8:
        print("Converged.")
        break

# 3. Compare the two solutions
print(f"||x_gn - x_ref|| = {norm(x - x_ref):.2e}")
```

4.4 Derivative Verification

The `getFiniteDifferenceA()` method provides a convenient way to verify the analytic Jacobian:

Listing 5: Comparing analytic and numerical Jacobians (`examples/05_derivative_verification.py`).

```python
import numpy as np
from scipy.sparse import coo_matrix
import pyLGC

ev = pyLGC.Evaluator("Title-Example.lgc2")
ok, sol = ev.tryLGCSolve()
ev.setParameters(sol)
ev.evaluate()

# Analytic Jacobian
r, c, v, nr, nc = ev.getAMatrix()
A_analytic = coo_matrix((v, (r, c)), shape=(nr, nc)).toarray()

# Numerical Jacobian (C++ internal finite differences)
A_fd = ev.getFiniteDifferenceA(1e-7)

# Compare
diff = np.abs(A_analytic - A_fd)
```

```
print(f"Max difference: {diff.max():.2e}")
print(f"Mean difference: {diff.mean():.2e}")
```

References

[1] J. Gutekunst, *LGC – Position Estimation Software: Principal Concepts and Methods*, CERN, 2024. https://edms.cern.ch/document/3067818/1

[2] M. Barbier, *Least Square Equations for LGC2*, CERN, 2015. https://edms.cern.ch/document/1475972/2

[3] D. Wells and E. Krakiwsky, *The Method of Least Squares*, University of New Brunswick, Department of Geodesy and Geomatics Engineering, Lecture Notes No. 18, 1971.